

VLLM专题（二十七）—使用 Kubernetes

原创

AI专题精讲

已于 2025-03-19 10:25:51 修改

阅读量57

收藏

点赞数

版权

分类专栏：

大模型专题系列

 文章标签：

kubernetes

贪心算法

算法

 大模型专题系列 专栏收录该内容

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益

在Kubernetes上部署vLLM是一种可扩展且高效的方式来服务机器学习模型。本指南将引导您使用原生Kubernetes部署vLLM。

此外，您还可以使用以下任意一种方式将vLLM部署到Kubernetes：

- Helm
- InftyAI/llmaz
- KServe
- kubernetes-sigs/lws
- meta-llama/llama-stack
- substratusai/kubeai
- vllm-project/aibrix
- vllm-project/production-stack

1. 前置条件

确保您已经拥有一个运行中的 Kubernetes 集群，并且集群中配置了 GPU。

2. 使用原生 Kubernetes 进行部署

1. 为 vLLM 创建 PVC、Secret 和 Deployment
- PVC 用于存储模型缓存，这是可选的。您也可以使用 hostPath 或其他存储选项。
- 以下是一个 Kubernetes PersistentVolumeClaim (PVC) 的配置示例，用于为 Mistral-7B 模型创建存储卷：

¥49.90
¥99.00

92 篇文章

已订阅

8折续

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mistral-7b
  namespace: default
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 50Gi
  storageClassName: default
  volumeMode: Filesystem
```

配置说明：

name: mistral-7b

PVC 的名称为 **mistral-7b**，您可以根据需要修改。

namespace: default

PVC 位于 **default** 命名空间，您可以根据需要更改为其他命名空间。

accessModes: ReadWriteOnce

存储卷的访问模式为 **ReadWriteOnce**，表示该卷只能被单个节点以读写模式挂载。

resources.requests.storage: 50Gi

请求的存储空间为 50Gi，您可以根据模型大小和需求调整。

storageClassName: default

使用的存储类名称为 **default**，如果您有自定义的存储类，可以替换为相应的名称。

volumeMode: Filesystem

存储卷的模式为文件系统（**Filesystem**），适用于大多数场景。

```
apiVersion: v1
kind: Secret
metadata:
  name: hf-token-secret
  namespace: default
type: Opaque
stringData:
  token: "REPLACE_WITH_TOKEN"
```

说明：

- **name: hf-token-secret** : Secret 的名称，可以根据需要修改。
- **namespace: default** : Secret 所在的命名空间，默认为 **default**，可以根据需要修改。
- **stringData** : 用于存储非加密的字符串数据，例如 Hugging Face 的访问token。
- **token: "REPLACE_WITH_TOKEN"** : 将 **REPLACE_WITH_TOKEN** 替换为实际的 Hugging Face 访问 token。

以下是使用 Kubernetes 部署 vLLM 模型服务器的示例，分别针对 NVIDIA GPU 和 AMD GPU 的配置。我们将以 Mistral-7B-Instruct-v0.3 模型为例。

NVIDIA GPU 部署示例

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mistral-7b
  namespace: default
  labels:
    app: mistral-7b
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mistral-7b
  template:
    metadata:
      labels:
        app: mistral-7b
    spec:
      volumes:
        - name: cache-volume
          persistentVolumeClaim:
            claimName: mistral-7b
        # vLLM needs to access the host's shared memory for tensor parallel inference.
        - name: shm
          emptyDir:
            medium: Memory
            sizeLimit: "2Gi"
      containers:
        - name: mistral-7b
          image: vllm/vllm-openai:latest
          command: ["/bin/sh", "-c"]
          args: [
            "vllm serve mistralai/Mistral-7B-Instruct-v0.3 --trust-remote-code --enable-chunked-prefill --max_num_batched_tokens 1024"
          ]
      ]
```

```

env:
- name: HUGGING_FACE_HUB_TOKEN
  valueFrom:
    secretKeyRef:
      name: hf-token-secret
      key: token
ports:
- containerPort: 8000
resources:
  limits:
    cpu: "10"
    memory: 20G
    nvidia.com/gpu: "1"
  requests:
    cpu: "2"
    memory: 6G
    nvidia.com/gpu: "1"
volumeMounts:
- mountPath: /root/.cache/huggingface
  name: cache-volume
- name: shm
  mountPath: /dev/shm
livenessProbe:
  httpGet:
    path: /health
    port: 8000
    initialDelaySeconds: 60
    periodSeconds: 10
readinessProbe:
  httpGet:
    path: /health
    port: 8000
    initialDelaySeconds: 60
    periodSeconds: 5

```

AMD GPU:

如果使用 AMD ROCm GPU（如 MI300X），您可以参考下面的 `deployment.yaml` 文件。

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: mistral-7b
  namespace: default
  labels:
    app: mistral-7b
spec:

```

```
replicas: 1
selector:
  matchLabels:
    app: mistral-7b
template:
  metadata:
    labels:
      app: mistral-7b
  spec:
    volumes:
      # PVC
      - name: cache-volume
        persistentVolumeClaim:
          claimName: mistral-7b
      # vLLM needs to access the host's shared memory for tensor parallel inference.
      - name: shm
        emptyDir:
          medium: Memory
          sizeLimit: "8Gi"
    hostNetwork: true
    hostIPC: true
    containers:
      - name: mistral-7b
        image: rocm/vllm:rocm6.2_mi300_ubuntu20.04_py3.9_vllm_0.6.4
        securityContext:
          seccompProfile:
            type: Unconfined
          runAsGroup: 44
          capabilities:
            add:
              - SYS_PTRACE
        command: ["/bin/sh", "-c"]
        args: [
          "vllm serve mistralai/Mistral-7B-v0.3 --port 8000 --trust-remote-code --enable-
chunked-prefill --max_num_batched_tokens 1024"
        ]
        env:
          - name: HUGGING_FACE_HUB_TOKEN
            valueFrom:
              secretKeyRef:
                name: hf-token-secret
                key: token
        ports:
          - containerPort: 8000
        resources:
          limits:
            cpu: "10"
            memory: 20G
```

```

      amd.com/gpu: "1"
    requests:
      cpu: "6"
      memory: 6G
      amd.com/gpu: "1"
    volumeMounts:
    - name: cache-volume
      mountPath: /root/.cache/huggingface
    - name: shm
      mountPath: /dev/shm

```

您可以从 ROCm/k8s-device-plugin 获取完整的示例、步骤和示例 YAML 文件。

2. 创建 vLLM 的 Kubernetes 服务

接下来，创建一个 Kubernetes 服务文件，用于暴露 mistral-7b 部署：

```

apiVersion: v1
kind: Service
metadata:
  name: mistral-7b
  namespace: default
spec:
  ports:
    - name: http-mistral-7b
      port: 80
      protocol: TCP
      targetPort: 8000
    # The label selector should match the deployment labels & it is useful for prefix caching feature
  selector:
    app: mistral-7b
  sessionAffinity: None
  type: ClusterIP

```

3 部署和测试

使用以下命令通过 `kubectl apply -f <文件名>` 应用部署和服务配置：

```

kubectl apply -f deployment.yaml
kubectl apply -f service.yaml

```

要测试部署，运行以下 `curl` 命令：

```
curl http://mistral-7b.default.svc.cluster.local/v1/completions \
-H "Content-Type: application/json" \
-d '{

    "model": "mistralai/Mistral-7B-Instruct-v0.3",
    "prompt": "San Francisco is a",
    "max_tokens": 7,
    "temperature": 0

}'
```

如果服务部署正确，您应该会收到来自 vLLM 模型的响应。

3. 结论

使用 Kubernetes 部署 vLLM 可以有效地扩展和管理利用 GPU 资源的 ML 模型。按照上述步骤，您应该能够在 Kubernetes 集群中设置和测试 vLLM 部署。



已关注



分享

专栏目录

已订阅

AI专题精讲

k8s 部署满血版Deepseek-R1

忘记旧账号的运维~ 773

Deepseek-R1 k8s 部署

在 K8s 上使用 vLLM 部署大型语言模型 (LLM)

iCloudEnd的博客 717

vLLM 是一个用于 LLM 推理和服务的开源库，提供快速实施和用户友好的功能。vLLM 由加州大学伯克利分校专家精心制作，并在 Chat bot Arena 和 Vicuna Demo 中使用，它拥有一系列功能，包括：性能增强：利用分页注意力.. 机制，与 Hugging Face Transformers 相比，vLLM 的吞吐量显着提高了 24 倍，确保了快速高效的 LLM 服务。用户友好的界面：vLLM 专注于无缝集成，通过其直观的界面简化了 LLM 部署。

VLLM 专题（二十四）—环境变量 最新发布

探索人工智能革命，深入算法原理与创新应用，未来... 72
技无限可能。

对于 Kubernetes 用户，请特别注意：不要将服务命名为 vllm，否则 Kubernetes 设置的环境变量可能与 vLLM 的环境变量发生冲突，因为 Kubernetes 会为每个服务设置以大写服务名为前缀的环境变量。请注意，VLLM_PORT 和.. VLLM_HOST_IP 设置的是 vLLM 内部使用的端口和 IP，而不是 API 服务器的端口和 IP。如果您使用 --host \$ VLLM_HOST_IP 和 --port \$ VLLM_PORT 启动 API 服务器，它将无法正常工作。